

Day 5 - Strings, Formatting, Files, and Context Managers

Day 5 — Strings, Formatting, Files, and Context Managers

Objectives

- Get comfortable with the string methods you'll use daily
- Write f-strings fluently, including controlling how numbers are displayed
- Read and write real files on disk, correctly
- Understand *why* the `with` keyword exists, not just that you should type it

Strings are immutable — a quick reminder, now with the methods that matter

Day 1 established that strings can't be changed in place — every string "modification" you do actually produces a brand new string, leaving the original completely untouched. Keep that in mind as you read the methods below: every single one of them **returns a new string** rather than changing the one you called it on.

```
s = " Hello, World! "
```



```
s.strip()          # "Hello, World!"      -- removes whitespace from both ends
s.lower()          # " hello, world! "    -- an all-lowercase COPY
s.upper()          # "  HELLO, WORLD!  "
s.replace("l", "L") # "  HeLlo, WorLd! "  -- replaces every occurrence
s.split(",")       # [' Hello', ' World! '] -- splits into a list wherever "," appears
",".join(["a", "b", "c"]) # "a,b,c" -- the REVERSE of split: glue a list of strings together
s.startswith(" He") # True
s.endswith("! ")    # True
"42".isdigit()      # True -- checks whether every character is a digit
s.find("World")      # 9 -- the index where "World" starts, or -1 if not found
len(s)              # the number of characters, including the spaces
```

Notice that none of these change `s` itself — if you want to keep the result, you must assign it, usually back to the same name:

```
s = " Hello, World! "
s = s.strip() # NOW s actually refers to the new, trimmed string
```

`.split()` — a common beginner trap

Calling `.split()` with **no argument at all** splits on *any run* of whitespace (spaces, tabs, newlines) and automatically discards empty results:

```
" hi there ".split() # ['hi', 'there']
```

But calling `.split(" ")`, giving it an explicit single space, splits on that *exact* literal character every time it appears — including when that produces empty strings from doubled-up spaces:

```
" hi there ".split(" ") # ['', '', 'hi', '', '', 'there', '', '']
```

These look similar but behave very differently — this exact mismatch is a genuinely common source of bugs when parsing text that might have inconsistent spacing (like typed-in user input, or lines from a file). When in doubt, prefer plain `.split()` with no argument unless you specifically need to split on one exact character (like a comma in a CSV line, which you'll see on Day 11).

f-strings — the modern way to build text that includes variable values

An **f-string** lets you embed the value of a variable (or any expression) directly inside a piece of text, by writing **f** right before the opening quote and wrapping the variable in **{curly braces}**:

```
name = "Ana"
score = 91.5

print(f"{name} scored {score}%")          # "Ana scored 91.5%"
print(f"{name} scored {score:.1f}%")      # "Ana scored 91.5%" -- .1f means "1 digit after the d
print(f"{score=}")                        # "score=91.5" -- shows the variable name AND its v
```

The part after a colon inside the braces (like `.1f` above) is called a **format spec** — it controls exactly how the value is displayed:

```
f"{3.14159:.2f}"          # "3.14" -- round to 2 decimal places
f"{1000000:,}"           # "1,000,000" -- insert thousands separators
f"{0.5:.0%}"             # "50%" -- format as a percentage
f"{ 'left':<10}|"         # "left      |" -- left-align within a width of 10 characters
f"{ 'right':>10}|"        # "          right|" -- right-align within a width of 10
```

You'll use f-strings constantly, in essentially every piece of Python code you write from here on — for printing results, building error messages, and formatting output for a person to read. They're the modern, preferred way to do this in Python; you may encounter two older styles in existing code you read (**%-formatting** and `.format()`) — recognize them if you see them, but write new code with f-strings.

Files — reading and writing data that outlives your program

Everything you've done so far has lived only in your program's memory while it runs, and vanished the moment it finished. A **file** lets your program save data permanently on disk, so it's still there the next time you run the program (or open it in another program entirely). Opening a file gives you a **file object** — a temporary handle Python uses to read from or write to that file.

```
with open("data.txt", "r") as f:
    contents = f.read()
print(contents)
```

`open("data.txt", "r")` opens the file named `data.txt` in **read mode** (`"r"`, the default if you don't specify a mode) and gives you back a file object, which this code names `f`. `f.read()` reads the *entire* file's contents into one big string.

Why `with`? — the problem it solves

Opening a file means your program is holding onto a limited resource that the operating system is tracking (only so many files can be open by a program at once, and other programs may be waiting to use the same file). You're supposed to **close** a file once you're done with it, to release that resource:

```
f = open("data.txt")
data = f.read()
f.close()          # if anything above this line raises an error, close() NEVER RUNS
```

The problem: if something goes wrong between `open()` and `close()` — say, `f.read()` raises an unexpected error — `f.close()` is skipped entirely, and the file is left open longer than it should be. Doing this by hand, reliably, in every possible situation (including ones you didn't anticipate) is genuinely hard to get right.

`with` solves this completely. It's called a **context manager**, and its entire job is: "run this setup, run my code, and no matter what happens — even if an error occurs partway through — guarantee the cleanup step runs afterward."

```
with open("data.txt") as f:
    data = f.read()
# by this point, f is GUARANTEED to be closed -- whether the block above succeeded or raised an error
```

The rule to internalize starting today: any time you open a file, always use `with`. You'll learn the mechanism behind exactly how `with` guarantees this (two special methods called `__enter__` and `__exit__`) on Day 9, once you've learned how to write your own classes — for now, just know that `with` is the correct, safe way to work with files, full stop.

Reading a file, a few different ways

```
with open("data.txt") as f:
    for line in f:          # loop directly over the file object -- gives you one line at a time
        print(line.strip()) # .strip() removes the trailing newline character each line ends w/ \n

with open("data.txt") as f:
    lines = f.readlines()   # reads the WHOLE file at once into a list of lines
```

Looping directly over the file object (the first example) is generally preferred for larger files, because it reads one line at a time instead of loading the entire file into memory at once — you'll see exactly why this matters when you meet generators on Day 10.

Writing to a file

```
with open("out.txt", "w") as f:    # "w" = write mode
    f.write("hello\n")             # \n is the special character meaning "start a new line"
```

Careful: "w" mode completely erases (truncates) the file first, if it already exists, before writing your new content — there's no confirmation prompt, no undo. If you instead want to add content to the *end* of an existing file without erasing what's already there, use `"a"` (append) mode instead:

```
with open("out.txt", "a") as f:
    f.write("more\n")
```

Summary of the modes you'll use most: `"r"` read (default), `"w"` write/erase-then-write, `"a"` append, and adding `"b"` to any of them (e.g. `"rb"`) switches to binary mode, for non-text files like images, where you get raw **bytes** back instead of text.

Encoding — a subtlety worth knowing about even as a beginner

Text files are actually stored as raw bytes on disk, and there are different systems ("encodings") for converting between bytes and human-readable characters. If you don't specify one, Python picks a default based on your operating system's settings — which can silently differ between your computer and someone else's, or a server's, causing a file that works fine for you to behave oddly for a teammate. The fix is simple: be explicit.

```
with open("data.txt", encoding="utf-8") as f:  
    contents = f.read()
```

"utf-8" is the overwhelmingly standard choice today and correctly handles virtually all text you'll encounter, including non-English characters and emoji — get in the habit of specifying it explicitly whenever you open a text file, starting with today's exercises.

Exercises

Open `exercises.py`, implement each `# TODO`, and run `python exercises.py`. This one creates and deletes a couple of small temporary files while it runs — that's expected and part of testing your file-handling code for real.